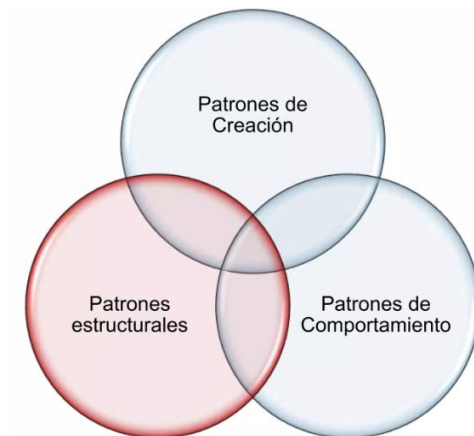




PDS-ESTRUCTURALES: BRIDGE

ING. ZOILA RUIZ, PHD

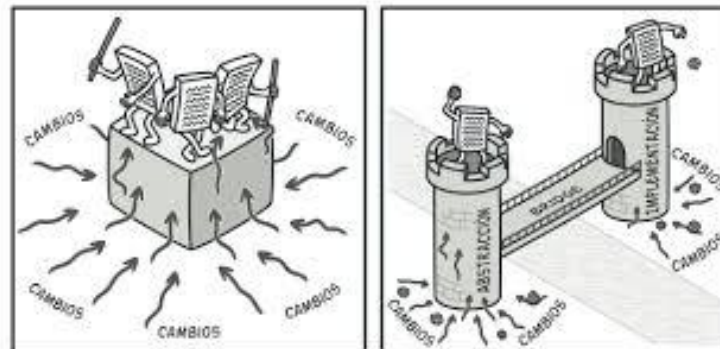
BRIDGE

Es un patrón de diseño estructural utilizado para desacoplar una abstracción de su implementación de manera que las dos puedan ser modificadas por separado sin necesidad de modificar la otra, o puedan variar de forma independiente.

Divide la lógica de negocio o una clase muy grande en jerarquías de clases separadas que se pueden desarrollar independientemente.

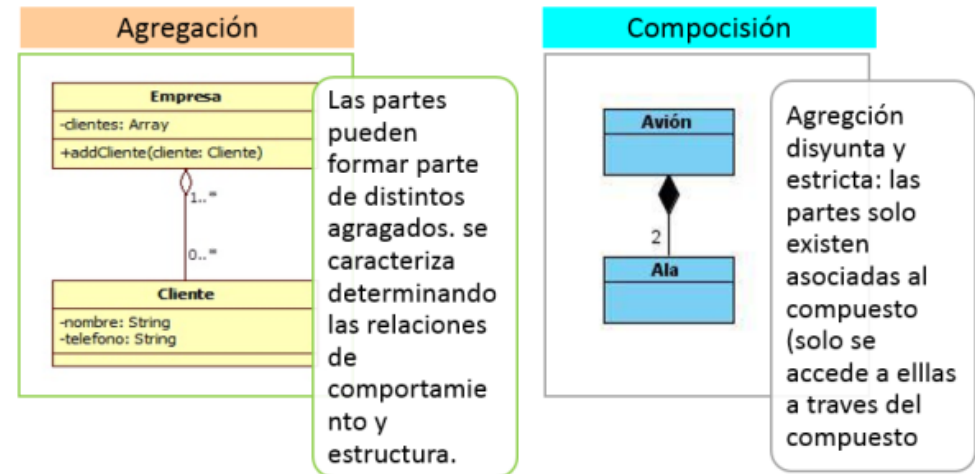
Las jerarquías en las que se dividen son conocidas como Abstracción e Implementación. La Abstracción obtendrá una referencia a un objeto de la segunda jerarquía (Implementación)

Este patrón permite mayor flexibilidad y reutilización del código.



BRIDGE

- ❖ Este patrón permite la creación de objetos relacionados a través de la agregación o composición en lugar de la herencia.
- ❖ La abstracción e implementación se modifican en tiempo de ejecución.
- ❖ Los cambios en la abstracción no afectan al cliente.
- ❖ Se extrae las dimensiones en jerarquías independientes



BRIDGE

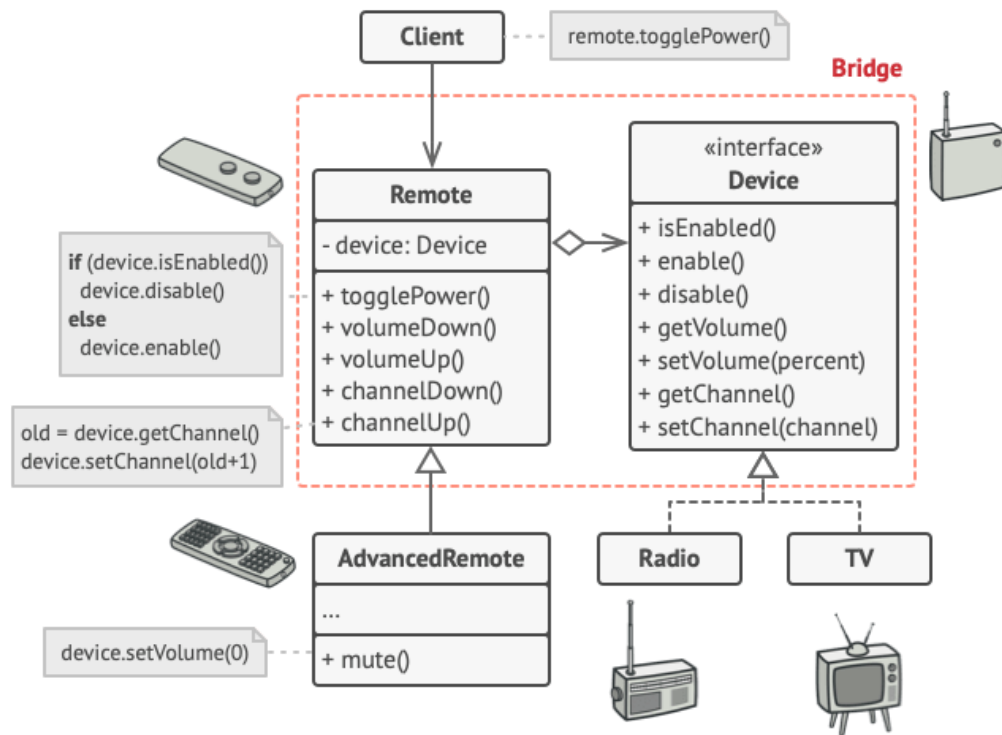
Contexto: Quieres evitar una explosión de clases cuando tienes múltiples dimensiones de variación (por ejemplo, formas y dispositivos).

Problema: ¿Cómo separar la abstracción de su implementación para que ambas puedan evolucionar independientemente?

Solución: Crear dos jerarquías separadas: una para la **abstracción** y otra para la **implementación**, y conectar ambas mediante una relación de composición.

CONSECUENCIAS POSITIVAS	CONSECUENCIAS NEGATIVAS
Separa construcción de representación	Mayor número de clases
El código cliente funciona con abstracciones de alto nivel. No está expuesto a los detalles de la plataforma.	Puede ser que el código se complique si aplicas el patrón a una clase muy cohesionada.
Principio de abierto/cerrado.	
Principio de responsabilidad única.	

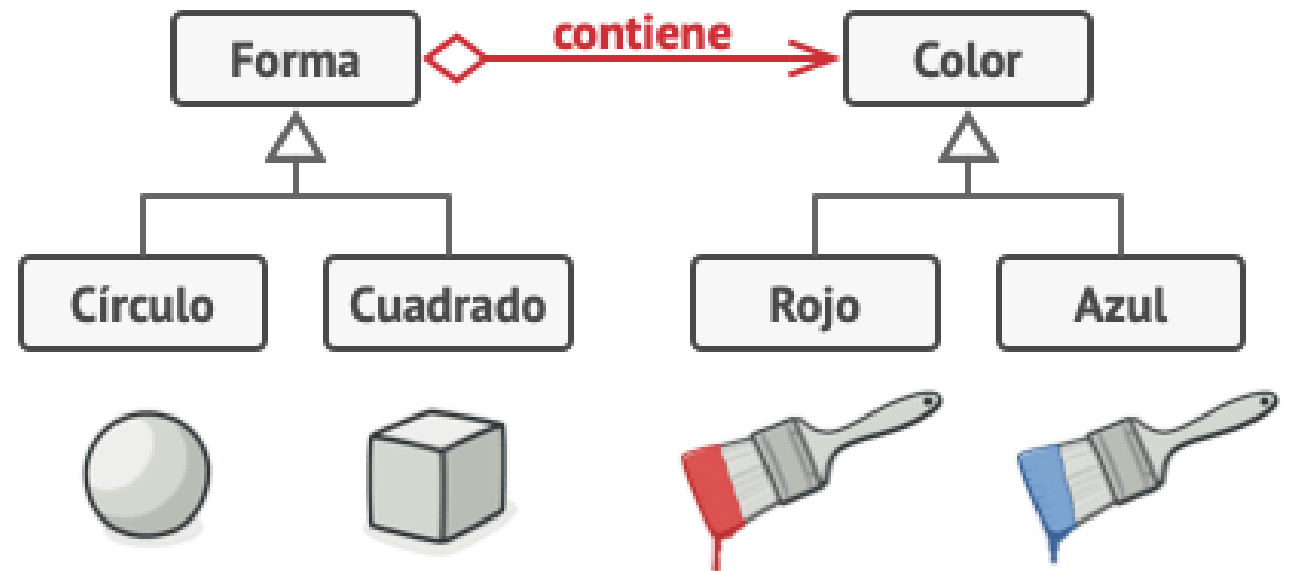
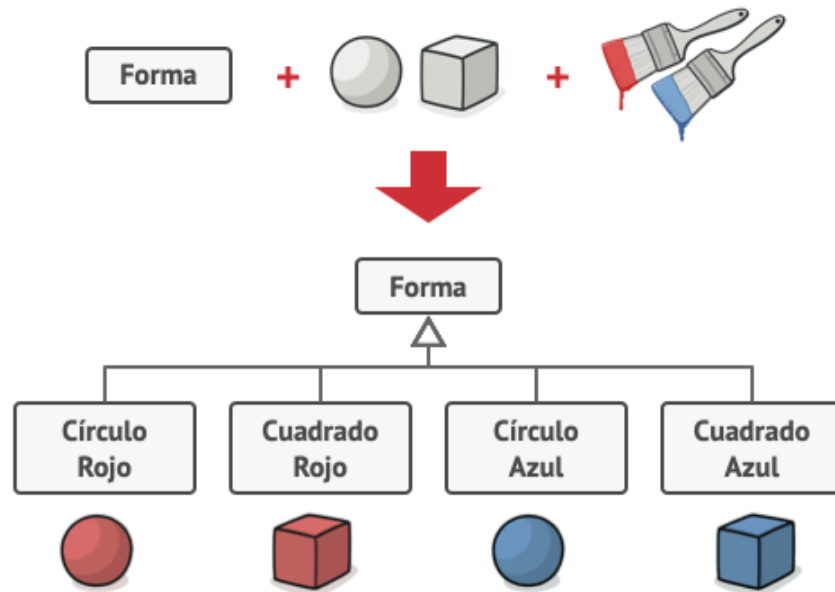
BRIDGE



Para la implementación del patrón se debe considerar los siguientes puntos:

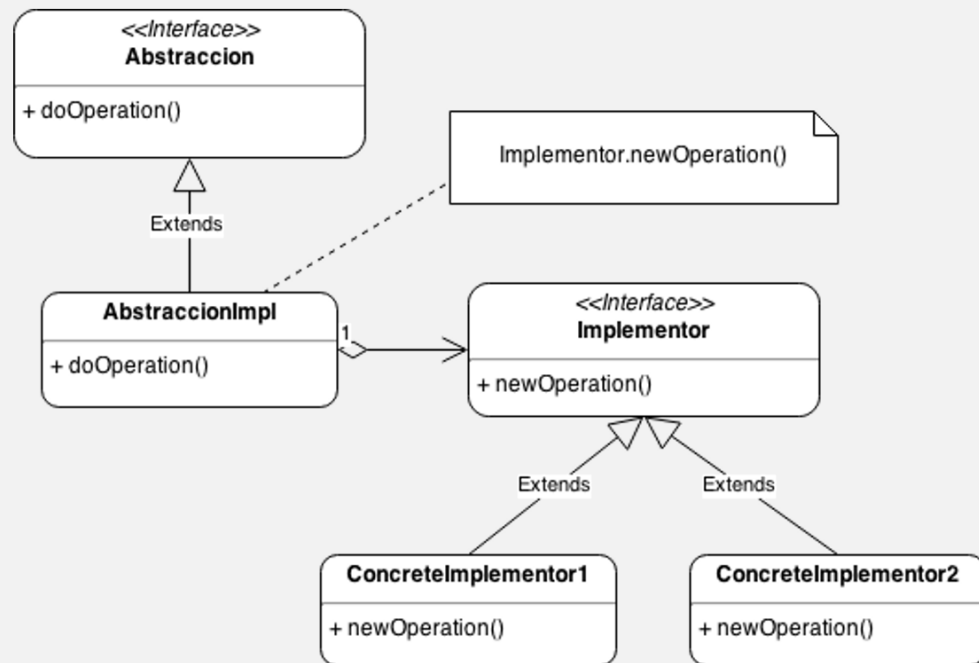
- ❖ Identificar las dimensiones de las clases
- ❖ Comprobar qué operaciones necesita el cliente
- ❖ Crear clases concretas de implementación
- ❖ Añadir un campo de referencia para el tipo de implementación.
- ❖ Pasar un objeto de implementación al constructor de la abstracción.

BRIDGE



BRIDGE

Bridge pattern – Class diagram



Los elementos de este patrón son:

Abstraccion: Interfaz que define la estructura de clase

AbstraccionImpl: Clase utilizada como puente para desacoplar la abstracción de la Implementación

Implementor: Define una interfaz para la Implementación (opcional)

Concret Implementor: clases concretas que son propensas a cambiar

BRIDGE

Usos:

Cuando se quiera dividir y organizar una clase monolítica que tenga muchas variantes de una sola funcionalidad.

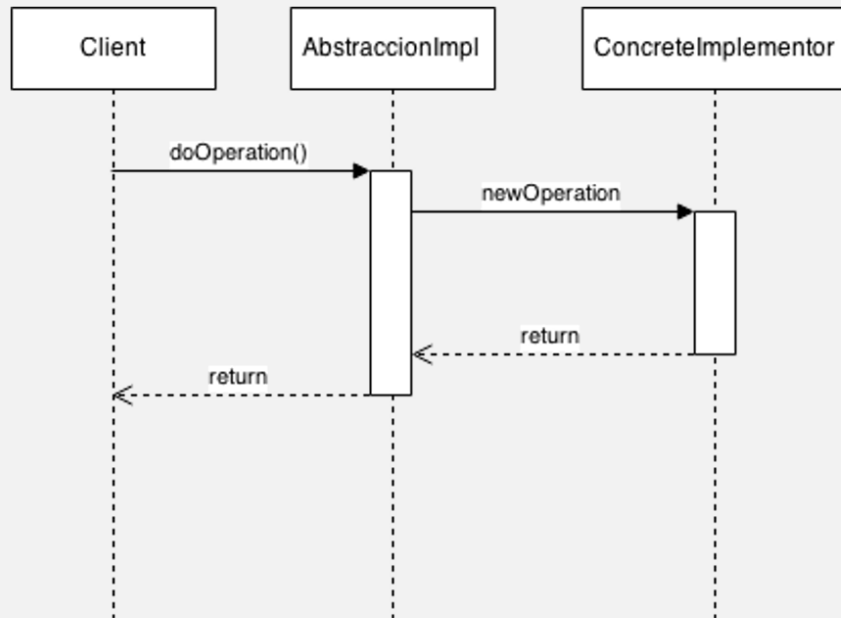
Cuando se necesite extender una clase en varias dimensiones independientes

Cuando se necesite poder cambiar implementaciones durante el tiempo de ejecución.

Aplicaciones multiplataforma donde la lógica de negocio y la interfaz gráfica deben variar de manera independiente.

BRIDGE

Bridge pattern – Diagram of sequence



- ❖ El cliente ejecuta una operación
- ❖ AbstraccionImpl replica la petición al ConcretImplementor
- ❖ El ConcretImplementor regresa los resultados
- ❖ El AbstraccionImpl convierte los resultados para que sean devueltos al cliente.